

Verification of the external audit findings (74 items) — PoD Inbox + Privacy Portal

Verification of 74 external audit findings · coti-pod-inbox-contracts @ 89121ad · coti-contracts @ 8a0c492 · gist 7aa7524c · 2026-09-07

Date: 2026-09-07 · **Verifier:** Claude (this session), with three independent sub-agents for the Low/Informational batches **Input:** the 74-item findings list you pasted (numbered here 1–74 in the order pasted; titles in `external-findings-verification/findings_index.txt`) and its PoC gist `kuldeep23907/a7f659352f240ca9e67f49ed3c8c3220` at commit `7aa7524c` (2026-09-06). **Code verified against:** `coti-pod-inbox-contracts` HEAD (= audited `89121ad` plus one comment line) and `coti-contracts` (byte-identical to audited `8a0c492`).

1. Bottom line

PoCs. Every gist test was compiled and executed against the real contracts with Foundry 1.5.1 / solc 0.8.28.

Scope	Suites	Result
A — <code>coti-pod-inbox-contracts</code> (Inbox, FeeManager, oracles)	20	108 / 108 pass (incl. the B11 test, whose harness was missing from the gist and was written here)
B — <code>coti-contracts</code> (portal, pToken, mother)	13 gist suites	77 pass, 2 fail by design (the two failing invariants are findings #6's evidence), 0 unexpected failures

Not runnable here: the A4 live COTI-testnet probe (not in the gist; needs the real MPC precompile). Everything else reproduced exactly as the auditor describes.

False positives. None of the 74 is a pure false positive: every item points at real code behaviour. Twelve contain a materially wrong sub-claim or an overstated impact (listed in §7), and one core claim (#32, precompile binds signatures to `tx.origin`) cannot be verified from either repository.

Severities. Of the 16 High/Medium items, 5 ratings stand and 11 are over-rated:

#	Claimed	Verified	Why
1	High	High	Unauthenticated, permissionless exact-balance read; confirmed in code and by PoC
2	High	Medium	Needs a lost COTI → source ack (trusted-miner liveness failure) <i>and</i> an admin kill; loss is recoverable only by admin rescue
3	High	Medium	Needs the admin to refund against explicit NatSpec instructions; the COTI Inbox's <code>incomingRequests[id].executed</code> is public, so the operator <i>can</i> check — the finding's "cannot be checked" is wrong
4	High (ack.)	High (ack.)	Trust model; matches our earlier POD-01
5	Medium	Medium	= our #1 (reverting release recorded <code>Success</code>)
6	Medium	Low	Admin-only, <code>whenPaused</code> , catastrophe path; = our #10 plus the (intended) fee clamp

#	Claimed	Verified	Why
7	Medium	Medium	Confirmed <i>and</i> made concrete: with the shipped Sepolia template, every encrypted <code>transfer / transferFrom</code> funded by <code>PodERC20.estimateFee()</code> reverts <code>CallbackFeeTooLow</code> (\$6)
8	Medium	Low	Fail-open is documented in <code>IPodPriceOracle</code> and the fee lib NatSpec; protocol revenue only; = our #11
9	Medium	Low	Pause = new-entry breaker by design (repo test proves it); a raced "rescue" pays entitled withdrawers
10	Medium	Low	Admin action; collateral recoverable via <code>adminRefundPendingDeposit</code> ; timer ordering is a valid config note
11	Medium	Low	Blacklist is documented as a portal entry-point control; one-hop bypass is a policy/scope gap, not a code defect
12	Medium	Low	User/admin address error; fee lost and request unterminalizable; no attacker, no fund loss beyond fees
13	Medium	Low	UX/availability; user re-submits; no loss
14	Medium	Medium	Matches our own oracle analysis (F-1/F-2/C-1); stale manual constant misprices every message
15	Medium	Low	Zero-tolerance quote is UX; overpayment bounded by <code>maxFee</code> ; <code>wrap</code> cap = our #13
16	Medium	Low	= our #3; admin-only, bounded (48 h), recoverable, and the finding itself shows the recovery

Low batch (17–32): all real; #21 and #30 are Informational (documented design / structurally unreachable).

Informational batch (33–74): all real; #39, #69, #70, #72 deserve Low (#34 Low unless the in-flight-blacklist policy is written down).

New finding from this verification (not in the list). The fee templates shipped in `scripts/deploy-utils.ts` set the COTI-side `constantFee` equal to `maxExecutionGas` (both 25,000,000). On a source chain the target leg is then admissible at exactly one gas figure, and with any realistic ETH/COTI price ratio (including the script's own testnet spot prices) no payment lands on it: every source → COTI two-way and one-way send reverts `TargetFeeTooLow` or `FeeGasTooHigh`. Even at 1:1 test prices, `PodERC20.estimateFee()` overshoots the cap by 300,000 units and reverts `FeeGasTooHigh`. Proven by a cross-repo Foundry test on the real Inbox + FeeManager + pToken (\$6). Severity **High (availability, configuration)** — conditional on the live Inbox carrying these template values, which is readable on-chain via `remoteMinFeeConfig()`.

2. How the PoCs were run

- Two Foundry workspaces were built from copies of each repo's `contracts/` tree (legacy `disperse/` dropped from B: pinned `=0.8.20`), gist tests placed at `test/foundry/`, gist mocks at `test/foundry/mocks/`, `forge-std` 1.16.2, OpenZeppelin 5.4.0 from each repo's `node_modules`, native solc 0.8.28. Scope A compiled with `shanghai` (the inbox repo's config), Scope B with `paris` (coti-contracts' config); `via_ir`, optimizer runs 1, both as the repos ship.
- `P14_PortalRescueSolvency.invariant.t.sol` imports `./PortalAccounting.invariant.t.sol`, which does not exist in either repo (the gist README says it is published as `P14F3_...`); that file was copied under the expected name. Neither repo contains any Foundry configuration or `test/foundry/` tree at the audited commits, contrary to the gist README's "the repository's existing Foundry configuration".

- `B11_TryDecodeWrap.t.sol` imports `./harness/B11Harness.sol`, absent from the gist. A 30-line harness (`test/foundry-external/harness/B11Harness.sol`) exposing `InboxBase._safeEncodeMethodCall` behind a raw-returning echo module was written; the 4 B11 tests then pass and confirm the wrap.
- The 7 failures in Scope B's log are `contracts/mocks/utis/mpc/OnBoard*TestsContract.sol` — the repo's own MPC mock test contracts that forge picks up and that need the real precompile. They are not gist tests.
- What is real vs modelled in the gist (their README is accurate on this): cross-chain *timing* claims (48 h expiry, retry) run against `MockCotiBridgeInbox`, a transcription of the Inbox; the mother's commit ordering (finding 2's debit-before-ack) runs the real `PodErc20CotiMother` under an MPC stub; P5's Inbox floor is a model of the dominant term only. `MockMpcPrecompile` tests logic, not cryptography, so nothing here speaks to the MPC layer's confidentiality itself.
- Invariant runs were shortened (32 runs × 64 depth) to fit the session; both designed-to-fail invariants still shrink to the sequences the README states (`deposit → rescueNative`; `deposit → withdraw → release → rescueERC20`).

Full per-test logs: `external-findings-verification/forge-scopeA-inbox-results.log`, `forge-scopeB-coti-contracts-results.log`, `forge-verifier-shipped-config-results.log`.

3. High and Medium findings (1–16)

Line numbers are HEAD line numbers; every cited line was read.

#1 — `estimateExecutionGasForMiner` is an unauthenticated confidential-balance oracle · claimed High → High, CONFIRMED

- `InboxMiner.sol:275-281`: `estimateExecutionGasForMiner` is `external override` with no `onlyMiner`, unlike `batchProcessRequests` (`:34-37`). Body `InboxEstimateGas.sol:74-141`: it copies the caller-supplied `mined.sourceContract` into `originalSender` (`:123-124`), and `_runIncomingExecution` (`InboxMiner.sol:329-333`) sets `_currentContext.remoteContract` from it, so `inboxMsgSender()` (`InboxBase.sol:356-360`) returns whatever the caller named. Nothing checks it against a trusted-remote set (the mother's `registered` mapping is `public`, `PodErc20CotiMother.sol:36`).
- `transferPublic` (`PodErc20CotiMother.sol:237`) → `_moveWithOptionalAllowance` with `amountIsPublic = true` and no allowance check → `if (!MpcCore.decrypt(MpcCore.ge(senderBalance, amount)))` raise, else respond (`:425-433`, `:482`). `respond` / `raise` are tagged and accumulated into `responseDataSize` / `errorDataSize` (`InboxEstimateGas.sol:46-64`, `InboxBase.sol:272`, `:505`) and returned in the terminal revert (`InboxEstimateGas.sol:140`).
- Gist PoC `EstimateGasBalanceOracle.t.sol`: 5/5 pass — non-miner reaches the handler, the size bit flips at the balance, binary search recovers the seeded 1,000,000 exactly, no state/event footprint, `staticcall` fails but `call` works. The live-testnet run (20 probes) is the auditor's claim; not reproducible here but every code step it depends on is confirmed.
- Severity: **High agreed**. Confidentiality of balances is the product; the read needs no key, no fee beyond gas, and is repeatable. Fix (add `onlyMiner`; make the two branches size-symmetric as defence in depth) is correct. Note the mother's own comment (`:416`) shows the authors considered probing only on the allowance path.

#2 — Committed COTI transfer can be cancelled on the source chain via `killStaleRequest` + `cancelFailedWithdrawal` · claimed High → Medium, CONFIRMED

- Mother writes both balances before asking: `_writeGarbledBalance(id, from, senderAfter)` (`:453`), recipient (`:479`), then `inbox.respond` (`:482`). `SystemFailed` is only written by `_handleSystemError`

(`PodERC20.sol:686-692`), reached only from `SYSTEM_SENDER` legs, which the Inbox mints only when the forward leg did not execute (`InboxBase.sol:565-617` , callers `InboxMiner.sol:209` , `:310` , `:346`). A lost ack therefore leaves the request `Pending` ; `killStaleRequest` (`PodERC20.sol:590-605`) is clock-only and writes `Failed` ; `cancelFailedWithdrawal` (`PrivacyPortal.sol:647-664`) accepts `Failed` or `SystemFailed` with no custody check; `_releaseWithdrawal` requires `Success` (`:861-864`), so the withdrawal is dead either way. All cited lines accurate. Gist `P7P9_DualTimerExit` (7/7) and `SystemFailedReachability` (5/5) pass; Q4 executes the wrongful cancel on the real portal.

- Why Medium, not High: the trigger is a return leg that is never delivered (or fails its first mine and is not retried within 48 h) — a liveness failure of the trusted miner, which the same report files as an acknowledged trust-model item — followed by an owner kill whose NatSpec (`IPodERC20.sol:298-305`) gives no COTI-side check. The user is harmed (pTokens debited on COTI, collateral locked) and only an admin `rescueERC20` while paused can return the collateral; the portal's COTI pTokens stay stranded (#72). Real, serious, but two failures of trusted parties away. Status "Resolved" in your list is not reflected in the audited code.
- Fix direction agreed: separate a far-side `Failed` (mother `raise`) from a locally written one, and do not let the kill act on a request whose forward leg may have executed.

#3 — `invalidatePendingRequest` creates permanent silent divergence / unbacked supply · claimed High → Medium, CONFIRMED (one detail wrong)

- `invalidatePendingRequest` (`PodERC20.sol:573-580`): minter-gated, no age gate, writes `Failed` ; `_setRequestStatus` (`:628-643`) rejects any later terminal write (`RequestNotPending`), so a late COTI `Success` callback reverts forever. `adminRefundPendingDeposit` (`PrivacyPortal.sol:614-643`) is `onlyFactoryAdmin whenPaused` with no age gate and only a `Success` check (`:627`). Gist `P21_PodErc20StateMachine` 3/3 pass. A COTI-side mint that lands after the refund is spendable there (the user can withdraw it through the portal), so the unbacked-supply consequence is real.
- Wrong detail: "no event" — `_setRequestStatus` emits `RequestStatusUpdated(requestId, Failed)` (`:642`); what is missing is a dedicated event like `StaleRequestKilled` . Overstated: "the operator is asked to resolve the difference by reading a confidential ledger by hand" — the COTI Inbox exposes `incomingRequests[id].executed` and `errors[id]` publicly (`InboxBase.sol:25` , `:23`), which is exactly the fact the NatSpec (`PrivacyPortal.sol:605-611`) tells the admin to establish.
- Why Medium: requires a privileged action against explicit documentation, with a public on-chain signal available; consequence (double payout) is severe, so not Low. Recommendation (age gate, event, reconcile-on-late-success) is sound; our validated fix #3/#8 already touches this path.

#4 — Centralization / trust model · claimed High (acknowledged) → High (acknowledged), CONFIRMED

- `MinerBase.sol` : flat `mapping(address => bool)` , single `onlyMiner` ; `batchProcessRequests` copies `sourceContract` , `isTwoWay` , `sourceRequestId` , selectors and fees verbatim (`InboxMiner.sol:104-119`), marks the named original request executed (`:142-150`), reject lane stores miner-supplied code/reason (`:80-91` , `:167-215`), TTL clock starts at ingest (`:111`). Gist `P2_ForgedRelease` (1/1), `A3_RejectLaneCensorship` (2/2), `E3_MinerAuthoritySurface` (2/2) pass. Matches our earlier `POD_INBOX_AUDIT.md` POD-01 ("fully trusted miner", Critical by design). The one untested claim (rejecting a two-way message with `isTwoWay=false` sends nothing) follows directly from `:208` . No severity change.

#5 — Reverting collateral release recorded as `Success` · claimed Medium → Medium, CONFIRMED (= our #1)

- `transferCallback` writes `Success` (`PodERC20.sol:336`) before the swallowed hook call (`:366-372`). Gist `P22_...` 2/2 and `P24_...` 4/4 pass (blocklisted recipient and rebasing collateral triggers). Their observation that

the withdrawal record rolls back to `TransferPending` and that a permissionless `triggerWithdrawalRelease` retry succeeds once the obstruction clears is correct and matches our fix (WETH fallback + `retargetStuckWithdrawal`). Severity agreed.

#6 — Rescue path: mutable `rescueRecipient`, `accumulatedPortalFees` clamp, `rescueERC20` has no reserve · claimed Medium → Low, CONFIRMED

- `feeRecipient` immutable, `rescueRecipient` settable (`PrivacyPortalFactory.sol:52-54`, `:423`); `rescueNative` clamps the fee counter (`PrivacyPortal.sol:748-766`); `rescueERC20` excludes only the `pToken` (`:769-782`). Both are `onlyFactoryAdmin nonReentrant whenPaused`. Gist invariants fail as designed (`P14: fees silently rewritten down`, `collateral leak via rescueERC20`).
- Why Low: this is the catastrophe path, admin-only, behind a visible pause; the clamp is the deliberate consistency rule you asked about earlier (fees can never exceed balance); the missing collateral reserve is our #10 (Low, fix `outstandingWithdrawalTotal` validated). The "trust placement" argument is a design observation, not a vulnerability.

#7 — Callback leg quoted from the ack size but validated against the request size · claimed Medium → Medium, CONFIRMED and sharpened

- Quote: `FeeManagerStubBase.sol:151-186` sizes the callback leg from `callbackMethodCallSize` and adds `callbackMethodExecutionGas` (`:181`). Validator: `FeeManager.sol:183` checks `callerGasLocalUnits < expectedMinFee(dataSize, localMin)` with `dataSize = abi.encode(methodCall).length` of the request (`InboxBase.sol:197`). Gist `C17_...` 9/9 pass, coefficient 1,200 gas/byte measured on the real fee path.
- Made concrete here (`test/foundry-external/verifier/ShippedConfigQuote.t.sol`, `MsgSizeProbe.t.sol`): `PodERC20.estimateFee()` quotes with fixed 512/512-byte legs and 300k exec gas each (`PodERC20.sol:39-45`, `:844-850`). Real message sizes: public mint 448 B, public transfer/approve 544 B, **encrypted transfer 768 B**, **encrypted transferFrom 864 B**. With the shipped Sepolia template (800 gas/byte, ×1.5 buffer, 100k callback gas, 256-byte error), the quote covers requests up to 762 B — so a public transfer funded by the quote is accepted, while encrypted `transfer(to, itUint256, cb)`, the auto-fee overload `transfer(to, itUint256)`, and encrypted `transferFrom` all revert `CallbackFeeTooLow(1_371_600)`. The confidential path is the product; the official estimator cannot fund it. Severity agreed. Fix: quote and validate from the same size, or derive the callback minimum from `callbackExecutionGas` only.

#8 — Portal fee oracle fails open · claimed Medium → Low, DESIGN CHOICE (documented) with valid observability notes (= our #11)

- `PrivacyPortalFeeLib.sol:79-97`: `@dev Falls back to fixedFee ... when either rate is zero`; `IPodPriceOracle.sol:9-10`: "Feed adapters never revert; return 0 when a feed is unset, stale, or failed. Zero rates make resolvePortalFee skip dynamic pricing". `PortalFeeOracle.setTokenPriceUSD` refuses zero (`:31-36`). Gist P5 Part 5 (6 tests) and `P6_OracleFailClosed_ScopeA` (3) pass and show exactly the documented behaviour plus the Inbox's `OraclePriceZero` contrast (`FeeManager.sol:245-257`).
- Valid additions worth keeping: `usedDynamicPricing` is dropped on the charge path (`PrivacyPortal.sol:917`), a percentage-only config charges zero during an outage, and `wrap` passes the floor straight through (`:492-500`). Revenue-only impact; our review already withdrew a strict-mode fix because it bricks the exit path during an oracle outage.

#9 — Pausing does not stop the release leg; rescue can be raced · claimed Medium → Low, DESIGN CHOICE + real observation

- Confirmed: pause gates only entry points (`_checkDepositsNotPaused` , `_checkWithdrawalsNotPaused`), `triggerWithdrawalRelease` (:667) and `_releaseWithdrawal` (:853) have none, `depositsPaused()` / `withdrawalsPaused()` both return `paused()` (`PrivacyPortalFactory.sol:443-450`). Gist `P24a/b` pass.
- Why Low: the repo's own tests establish the pause as a new-entry circuit breaker (our #2 pause-half verdict); a "raced" release pays a `Success` withdrawal to its rightful recipient. The scenario where those releases *should not* be paid (forged `Success`) requires the trusted miner. The recommendation to give the rescue path a precondition that actually implies quiescence is reasonable.

#10 — `requestKillMinAge` (1 day) matures before `maxMessageLife` (48 h) · claimed Medium → Low, CONFIRMED

- `requestKillMinAge = 1 days` (`PodERC20.sol:556`), `DEFAULT_MAX_MESSAGE_LIFE = 172_800` (`FeeManager.sol:22`), monotonic status (:628-637), `refundFailedDeposit` needs `SystemFailed` (`PrivacyPortal.sol:592`), `adminRefundPendingDeposit` still works after a kill (:626-634 : with status `Failed` the minter-only invalidate is skipped). Gist P7 A1–A5 pass (expiry step modelled). Effect: a permissionless recovery becomes an admin one; no loss. Config ordering recommendation is sound.

#11 — Blacklist not applied to the pToken · claimed Medium → Low, CONFIRMED as a documented scope gap

- `IPrivacyPortalFactory.sol:23-26` : "Factory-level address blacklist read by portal clones before user entry points ... blocked from deposits and withdrawals on factory portals". No blacklist reference anywhere under `contracts/pod/token/perc20/` . Gist P10 C1–C4 pass (entry points gated; direct `PodERC20.transfer` bypasses; no freeze/clawback). Whether the confidential asset itself must enforce a sanctions list is a compliance decision; as code it does what its documentation says.

#12 — Codeless target treated as successful execution · claimed Medium → Low, CONFIRMED

- Raw `call` (`InboxBase.sol:799`) returns success for a codeless address; `executed = true` unconditionally (`InboxMiner.sol:382`); error entry only on failure (:384-395); retry and TTL both gated on `ERROR_CODE_EXECUTION_FAILED` (:303-311). No `code.length` check on the message path; the oracle path has one (`ChainlinkFeedLib.sol:19` , :61). Gist `A1` 2/2 pass. Consequence: fee lost, request unterminalizable, source app waits forever; for the portal only an admin-misconfigured `cotiMotherContract` (#58) reaches it, recoverable via `adminRefundPendingDeposit` . One-line fix (`if (targetContract.code.length == 0)` → record `EXECUTION_FAILED`) is correct.

#13 — Fee quote unreliable when gas price moves · claimed Medium → Low, CONFIRMED

- Quote takes an unclamped `gasPrice` (`FeeManagerStubBase.sol:151-157`), `PodERC20` passes `tx.gasprice` (:849); validation uses `_referenceGasPrice = basefee + minPriorityFeeWei` floored at 2 gwei, ceiling unset (`FeeManager.sol:227-243` , :14). Gist `C11` 4/4 pass. Availability only; user retries. (Their `maxGasPriceWei` discussion is accurate: `ensureDefaults` never sets it, no script calls `setGasPriceBounds` .) The "third manifestation" (quote exceeding `maxExecutionGas`) turns out to be the default state of the shipped templates — see §6.

#14 — Cross-chain conversion price is a manual constant with no age bound; freshness signals overstate · claimed Medium → Medium, CONFIRMED

- `getPricesUSD` returns the cache with no age check (`PriceOracle.sol:160-162`); `refreshCache` is public, advances `lastFetchTimestamp` before pulling (`:124-143`), retains the old value on a zero pull (`:146-157`); `fetchGateOpen` public (`:186`); `PoDPriceOracle.setTokenPriceUSD` stamps nothing (`:86-95`), `getLivePrices` prefers the manual peg (`:62-83`). Gist `D12` 5/5 and `D11D2` 5/5 pass. This is the same territory as our `ORACLE_SECURITY_ANALYSIS.md` (F-1, F-2, C-1, C-3 — rated High/Medium there), so Medium is consistent. Their ordering advice on `maxStaleness` vs feed publication time is correct.

#15 — Portal fee re-derived at execution with no tolerance; two-layer squeeze; `wrap` uncapped · claimed Medium → Low, CONFIRMED

- `_validateAndCollectPortalFee` (`PrivacyPortal.sol:888-897`) recomputes the floor and keeps any excess up to `maxFee`; `deposit` requires `msg.value > portalFee` and forwards the rest (`:404-408`); `wrap` passes the floor through (`:492-500`); portal reads `getLivePrices` (manual peg first) while the Inbox reads the cache (`PoDPriceOracle.sol:62-83` vs `PriceOracle.sol:160`). Gist `P5` 15/15 pass (Inbox floor modelled). Impact is rejection or bounded overpayment; `wrap` is our #13 (Low, `wrapWithMaxFee` validated). Their two unsettled questions (are both layers on one oracle; are the native-token addresses equal) are deployment facts, unverifiable here.

#16 — Remount rotates the minter the old portal's refund still needs · claimed Medium → Low, CONFIRMED (= our #3)

- `createPortalWithExistingPToken` requires only `paused()` (`PrivacyPortalFactory.sol:653`), retires, clones, pauses the clone, then `setMinter(portal)` (`:665-674`); `adminRefundPendingDeposit` → minter-only `invalidatePendingRequest` reverts `OnlyMinter`. Gist `P17` 8/8 pass. Bounded to a ≤48 h freeze with a permissionless refund after expiry, plus the two-step admin recovery (kill, then refund) the finding itself documents. Our validated fix lets the owner/factory invalidate.

4. Low findings (17–32) — sub-agent verification, spot-checked

Full evidence with line citations: `external-findings-verification/agent-low.md` .

#	Finding	Verdict	Severity	Notes
17	1/1 gas-price skew shipped; safeguards absent	Partially confirmed	Low	Config facts true; the "silent busy return leg" narrative is wrong — reply legs reuse the source-computed <code>callerFee</code> (<code>InboxBase.sol:279</code>), COTI's remote template prices nothing in scope
18	pToken adoption doesn't check underlying correspondence	Confirmed	Low	= our #4 residual; example direction in the text is inverted; fix (read old portal's <code>underlyingToken()</code>) sound
19	Deposits accepted before COTI registration	Confirmed	Low	= our #5; NatSpec already says "or immediately set"; missed residue: a failed, unretried registration can never be re-issued through the same factory
20	Two-way triple not re-validated at ingest	Confirmed	Low	Sub-claim wrong: a raise payload delivered to <code>transferCallback</code> reverts on the 11-word decode; the fail-open case is <code>approveCallback</code>

#	Finding	Verdict	Severity	Notes
21	Delegate-call modules irreplaceable	Confirmed	Informational	Documented design (<code>Inbox.sol:15-16</code>); the <code>Inbox address</code> is rotatable everywhere, only <code>cotiChainId</code> is fixed
22	Fee floor/cap in wei vs USD fee	Confirmed	Low	Bounds drift with native price; <code>usedDynamicPricing</code> dropped on charge path
23	Raw <code>setPTokenMinter</code> / <code>transferPTokenOwnership</code> bypass the remount sequence	Confirmed	Low	Our #4 fix guards <code>setPTokenMinter</code> ; <code>transferPTokenOwnership</code> remains raw; gist exercises only the ownership path
24	Mother <code>renounceOwnership</code> not blocked	Confirmed	Low	Every sibling overrides it; mother and all oracles do not
25	Lost burn ack never resolvable	Partially confirmed	Low	<code>killPTokenStaleRequest</code> → <code>Failed</code> → <code>finalizeBatchBurn</code> clears the reservation; only <code>pendingBurnAmount</code> stays inflated
26	Config change strands in-flight messages; one fails a batch	Partially confirmed	Low	Batch atomicity true; sub-claim (ii) wrong — retry re-checks no cap; sharper variant: lowering the <i>remote</i> cap below a stored <code>callerFee</code> also blocks the TTL callback
27	Non-reproducible build; dependency diverged	Partially confirmed	Low	<code>file:</code> dependency and no integrity confirmed; lockfile does record a version (0.0.1); npm-registry claims unverifiable
28	Shanghai config not COTI-deployable	Partially confirmed	Low	Config contradiction confirmed (<code>hardhat.config.ts:92-94</code> shanghai vs coti-contracts paris "COTI rejects PUSH0"); node behaviour and byte counts unverifiable
29	Skew ratio unbounded	Confirmed	Low	Validation gap only; 65535/1 test is not in the gist
30	Fee-admin auth in override; bare <code>InboxBase</code> deployable	Partially confirmed	Informational	A bare <code>InboxBase</code> is inert: <code>feeManager == 0</code> → <code>ModuleNotConfigured</code> on every forwarder; "stray value unrecoverable" contradicts the finding's own <code>collectFees</code> observation
31	Init preconditions not enforced on-chain	Confirmed	Low	Scripts close both gaps; the proposed constructor-sender check is unsafe under CREATE3 (creator is a proxy) — use <code>msg.sender == CREATEX</code> in <code>init</code>
32	Confidential input must be signed by the miner (<code>tx.origin</code>)	Not verifiable	Low (conditional)	Code path confirmed (validation runs inside the miner's tx via delegatecall); the precompile's signer binding is not documented in either repo

5. Informational findings (33–74) — sub-agent verification, spot-checked

Full evidence: `external-findings-verification/agent-info1.md` (33–53), `agent-info2.md` (54–74).

#	Finding	Verdict	Severity	Notes
33	<code>tokenPriceUpdatedAt</code> written, never read	Confirmed	Informational	Manual peg on <code>PoDPriceOracle</code> also stamps nothing
34	In-flight withdrawals ignore later blacklisting, undocumented	Confirmed	Low unless policy documented	= our #2 (Low)
35	Transaction graph public	Confirmed	Informational	Indexed on both chains (<code>TransferCompleted</code> on COTI too)
36	Retry success uses a different event	Confirmed	Informational	Already stated in NatSpec three times
37	<code>inboxErrorType()</code> misclassifies success as <code>Exception</code>	Confirmed	Informational	Documented (<code>IInbox.sol:12</code>); their fix (use the leg's own selector) cannot separate respond from raise and conflicts with #50's fix
38	Execution failure never notifies the source	Confirmed	Informational	Expiry notification is not automatic either (needs a <code>retryFailedRequest</code> call, <code>errorSelector≠0</code> , <code>callerFee≠0</code>)
39	Documented pre-funding flow doesn't exist; balance unrecoverable	Confirmed	Low	NatSpec invites operators to send native that no path can ever spend or sweep
40	Duplicate transfer selectors; one uncalled	Confirmed	Informational	<code>transferOwner</code> is labelled legacy in the interface
41	Burn nonce written after <code>respond</code>	Confirmed	Informational	Four sites, not two; no external call leaves the Inbox
42	Encrypted zero never read	Partially confirmed	Informational	It is read: emitted in <code>Transfer</code> and hashed into the ERC-7984 handle
43	<code>callerContract == originalSender</code> everywhere	Confirmed	Informational	Extra slot per request
44	Batch copied to memory before validation	Partially confirmed	Informational	<code>:61</code> is an alias, not a copy; interface lives in Scope B
45	<code>uint16</code> event counts truncate	Confirmed	Informational	Bounded by <code>PROTOCOL_MAX_METHOD_CALL_BYTES</code> ; widening changes topic0
46	Local/remote naming inversion	Confirmed	Informational	Already commented at <code>InboxMiner.sol:46-47</code>
47	Dead code (7 items)	Confirmed	Informational	Item (b) <code>_staticModule</code> is documented, not useless; (g) duplicate check is safe defence in depth
48	Unsafe <code>_encodeMethodCall</code> kept	Confirmed	Informational	Uncalled
49	Storage pointer taken before slot written	Confirmed	Informational	Their fix as phrased does not compile (memory struct → storage pointer)
50	Reply leg carries the other contract's error selector	Confirmed	Informational	Fix conflicts with #37's fix if both applied

#	Finding	Verdict	Severity	Notes
51	Gas budget named as fee	Confirmed	Informational	Interface NatSpec already says "not wei"
52	Returndata bounded only for the untrusted target	Confirmed	Informational	All unbounded callees are owner-configured
53	"Never reverts" feed-lib doc	Confirmed	Informational	Literally true; gas note valid
54	Escrow Failed unreachable	Confirmed	Informational	Keep enum ordinal
55	Stray native unattributable	Confirmed	Informational	2300-gas caveat on the proposed receive guard
56	"bps" is parts-per-million	Confirmed	Informational	Documented in the interface, not on the setters
57	Paid round trips that cannot succeed	Confirmed	Informational	
58	configureRouting lacks code check	Partially confirmed	Informational	Impact wrong: high-level calls to a codeless inbox revert (fail closed); constructor and configurePToken lack the check too
59	Capped returndata discards full length	Confirmed	Informational	Their critique of our earlier keccak suggestion is right
60	Context slot protection	Partially confirmed	Informational	_reply does assert context and caller; only a reentrancy guard is absent; EIP-1153 not deployable pre-Cancun
61	Invariants by convention (struct layouts, encodings, 1e18)	Confirmed	Informational	
62	Clearing a manual price offlines the lane	Partially confirmed	Informational	"No way to force recovery" is wrong: setLocal/RemoteTokenPriceUSD writes the cache in one call (= our oracle C-2)
63	Reject path skips ingest caps	Confirmed	Informational	
64	_tryDecodeAbiBytes add wraps	Confirmed	Informational	Reproduced with the harness written here; needs a hostile write-once module
65	63/64 gas shortfall; event reports budget	Partially confirmed	Informational	Mechanism true; magnitudes depend on off-repo miner tooling; "a sixty-fourth of the budget" is loose
66	Collateral measured in, trusted out	Confirmed	Informational	Consequences already scored (our #1, #10)
67	maxReplyMethodCallBytes unbounded	Confirmed	Informational	Bounded downstream
68	Factory quote surface ignores overrides	Confirmed	Informational	
69	Per-chain lanes, global fee config	Confirmed	Low	Multi-lane COTI Inbox is documented in-repo (Sepolia + Fuji); one remote leg/skew misprices the second lane

#	Finding	Verdict	Severity	Notes
70	Remount silently drops portal config	Confirmed	Low	Limits reset to unbounded, blacklist emptied, overrides dropped; nothing enumerates what to restore
71	Native deposit refunded wrapped	Confirmed	Informational	
72	pTokens sent to the portal are stranded	Confirmed	Low	Irreversible, no reconciliation, inflates nothing but is unburnable
73	<code>transferAndCall</code> makes <code>onlyPToken</code> non-boundary	Confirmed	Informational	Precondition weaker than stated (zero-balance encrypted transfer still fires the hook); harmless today
74	Stale mappings after <code>transferPTokenOwnership</code>	Partially confirmed	Informational	Mappings are not inert: they block <code>createPortal</code> for that underlying; deleting them is not obviously safe

6. New finding — shipped fee templates leave no admissible target payment (verifier-found)

Where. `scripts/deploy-utils.ts:365-403` (`FEE_CONFIG_COTI_SIDE` : `constantFee = maxExecutionGas = PROTOCOL_MAX_EXECUTION_GAS = 25_000_000` ; used as the *remote* template on every source chain via `testnetMinFeeConfigsForChain` , `:447-460` , applied by `applyTestnetFeeConfigs` , `:684-701`); `FeeManager.sol:183-190` (`TargetFeeTooLow` when units `< expectedMinFee = constantFee`); `InboxBase.sol:497-499` (`FeeGasTooHigh` when units `> remoteMax.maxExecutionGas`); `FeeManagerStubBase.sol:151-186` (quote adds `remoteMethodExecutionGas` on top of the constant); `PodERC20.sol:39-45` , `:844-850` (`estimateFee` adds 300,000).

Mechanism. The target leg's admissible gas is the closed interval `[constantFee, maxExecutionGas]` = `{25,000,000}` exactly. The units a payment buys are `floor((wei / gasPrice) × localPrice / remotePrice)` (`FeeManager.sol:181`). With price ratio `r` , only multiples of `r` (rounded down) are reachable; 25,000,000 is reachable only if some integer `k` gives `floor(k·r) = 25,000,000` .

- ETH \$3000 / COTI \$0.05 (`r = 60,000`): unreachable — every `k` gives `TargetFeeTooLow` or `FeeGasTooHigh` .
- The script's own testnet spot prices (`TESTNET_ETH_USD = 2103.41` , `TESTNET_COTI_USD = 0.01272522` , `r ≈ 165,296.9`): `k = 151 → 24,959,482` (too low), `k = 152 → 25,124,777` (too high). Unreachable for two-way sends **and** for the one-way `sendOneWayMessage` that `createPortal` uses to register a token on COTI.
- 1:1 prices (what the repo's own Hardhat tests use): reachable only by hand-paying exactly `25,000,000 × gasPrice`; `PodERC20.estimateFee()` (constant + 300,000) still reverts `FeeGasTooHigh(25_300_000, 25_000_000)` .

Evidence. `test/foundry-external/verifier/ShippedConfigQuote.t.sol` , run on the real `Inbox` + `FeeManager` + `PriceOracle` + `PodErc20Mintable` (log: `external-findings-verification/forge-verifier-shipped-config-results.log`):

```
[PASS] test_shipped_estimateFee_mint_reverts_FeeGasTooHigh      FeeGasTooHigh(25_320_000,
25_000_000) at ETH/COTI 3000/0.05
[PASS] test_shipped_constantOnlyTargetLeg_accepted_only_with_1to1_prices
[PASS] test_shipped_realisticPrices_noAdmissibleTargetPayment    k = 400..430 all revert
TargetFeeTooLow | FeeGasTooHigh
[PASS] test_shipped_testnetPrices_noAdmissiblePayment_twoWay_and_oneWay    k = 140..165 all
revert; units at k=151/152 = 24,959,482 / 25,124,777
[PASS] test_callbackLeg_encryptedTransfer_reverts_CallbackFeeTooLow (finding #7 made concrete)
```

Why the repo's tests do not see it. `FeeTemplateConstantFee.ts` only asserts the template invariants (`constantFee == maxExecutionGas`); every Inbox test that sends uses 1:1 oracle prices (`10n ** 18n` both legs) and hand-computed payments.

Severity. High as an availability defect of the shipped configuration: no user or portal on a source chain can send anything to COTI, and `estimateFee()` is unusable even where a payment exists. It is configuration, not contract logic, and it is correctable by the owner with one `updateMinFeeConfigs` call. **Caveat:** whether the live Sepolia/Fuji Inboxes carry these exact values is not verifiable from the repositories; `remoteMinFeeConfig()` on the deployed Inbox answers it (constant 25,000,000 with cap 25,000,000 confirms).

Fix. Set `constantFee` strictly below `maxExecutionGas` with a margin at least `remoteMethodExecutionGas` (300,000) plus the largest rounding step `localPrice/remotePrice` you expect ($\approx 170,000$ units at testnet prices, more if ETH/COTI diverges); or make the validator accept `min(units, maxExecutionGas)`; and add a test that quotes through `PodERC20.estimateFee()` and then sends, with non-trivial prices.

7. Corrections to the pasted text (facts that are wrong or overstated)

- #3 "no event" — `RequestStatusUpdated` is emitted; "cannot be checked on COTI" — the COTI Inbox's `incomingRequests` / `errors` are public.
- #17 reply legs are not re-priced by COTI's remote template; they reuse the source-computed `callerFee`.
- #20 a raise payload delivered to `transferCallback` reverts (11-word decode); the fail-open selector-equality case is `approveCallback`.
- #25 a killed burn request resolves through `finalizeBatchBurn`'s `Failed` branch; only `pendingBurnAmount` stays inflated.
- #26 (ii) retries re-check no admission cap; the real variant is the *remote* cap blocking reply and TTL legs.
- #27 the lockfile records version 0.0.1 for the linked dependency (no commit, no integrity).
- #30 (b) a bare `InboxBase` cannot be used: every fee forwarder reverts `ModuleNotConfigured`.
- #31 a constructor-`msg.sender` check is unsafe under CREATE3; gate `init` on the CreateX address instead.
- #42 the encrypted zero is emitted in `Transfer` and folded into the ERC-7984 handle.
- #44 `MinedRequest` memory `minedRequest = mined[i]` aliases; it does not copy.
- #49 the proposed fix (assign a memory struct to a storage pointer) is not valid Solidity.
- #58 a codeless inbox makes `createPortal` /pToken sends revert loudly (high-level calls), not "appear dispatched".
- #62 the price admin restores a cleared lane in one call via `setLocal/RemoteTokenPriceUSD`.
- #73 no stranded pTokens are needed to fire the hook (zero-balance encrypted transfer still reports `Success`).
- #74 the stale mappings are not inert; they block `createPortal` for that underlying.
- Gist README** "runs under the repository's existing Foundry configuration" — neither repo has any Foundry configuration or `test/foundry` tree at the audited commits; `PortalAccounting.invariant.t.sol` and `harness/B11Harness.sol` are not in either repo or the gist.

17. **#37 vs #50** the two recommended fixes conflict: applying both makes every reply leg `NotErrorContext` and breaks `PodERC20`'s error handlers.

8. Reproduce

```
# tools (github.com is reachable; binaries.soliditylang.org is not)
curl -L https://github.com/foundry-
rs/foundry/releases/download/stable/foundry_stable_linux_amd64.tar.gz | tar xz # forge 1.5.1
git clone https://gist.github.com/a7f659352f240ca9e67f49ed3c8c3220.git gist && git -C gist
checkout 7aa7524c
git clone --depth 1 https://github.com/foundry-rs/forge-std.git
# workspace A: cp -r coti-pod-inbox-contracts/contracts ffA/contracts; gist Scope-A *.t.sol ->
ffA/test/foundry/;
# test/foundry-external/harness/B11Harness.sol -> ffA/test/foundry/harness/;
verifier/ShippedConfigQuote.t.sol -> ffA/test/foundry/verifier/
# workspace B: cp -r coti-contracts/contracts ffB/contracts (rm -r ffB/contracts/disperse); gist
Scope-B *.t.sol + mocks/ -> ffB/test/foundry/;
# cp P14F3_PortalAccountingRescue.invariant.t.sol
ffB/test/foundry/PortalAccounting.invariant.t.sol; verifier/MsgSizeProbe.t.sol ->
ffB/test/foundry/verifier/
# configs: test/foundry-external/foundry.scopeA.toml / foundry.scopeB.toml (replace <SCRATCH> and
the solc path)
cd ffA && forge test -vv ; cd ../ffB && forge test -vv --no-match-path 'contracts/mocks/**'
```

Scope split by import: files importing `../../contracts/Inbox.sol`, `fee/*`, `MpcAbiReEncode.sol`, `MinerBase.sol`, `B11_*`, `F10_*`, `P6_OracleFailClosed_ScopeA` → A; files importing `../../contracts/pod/*` → B.